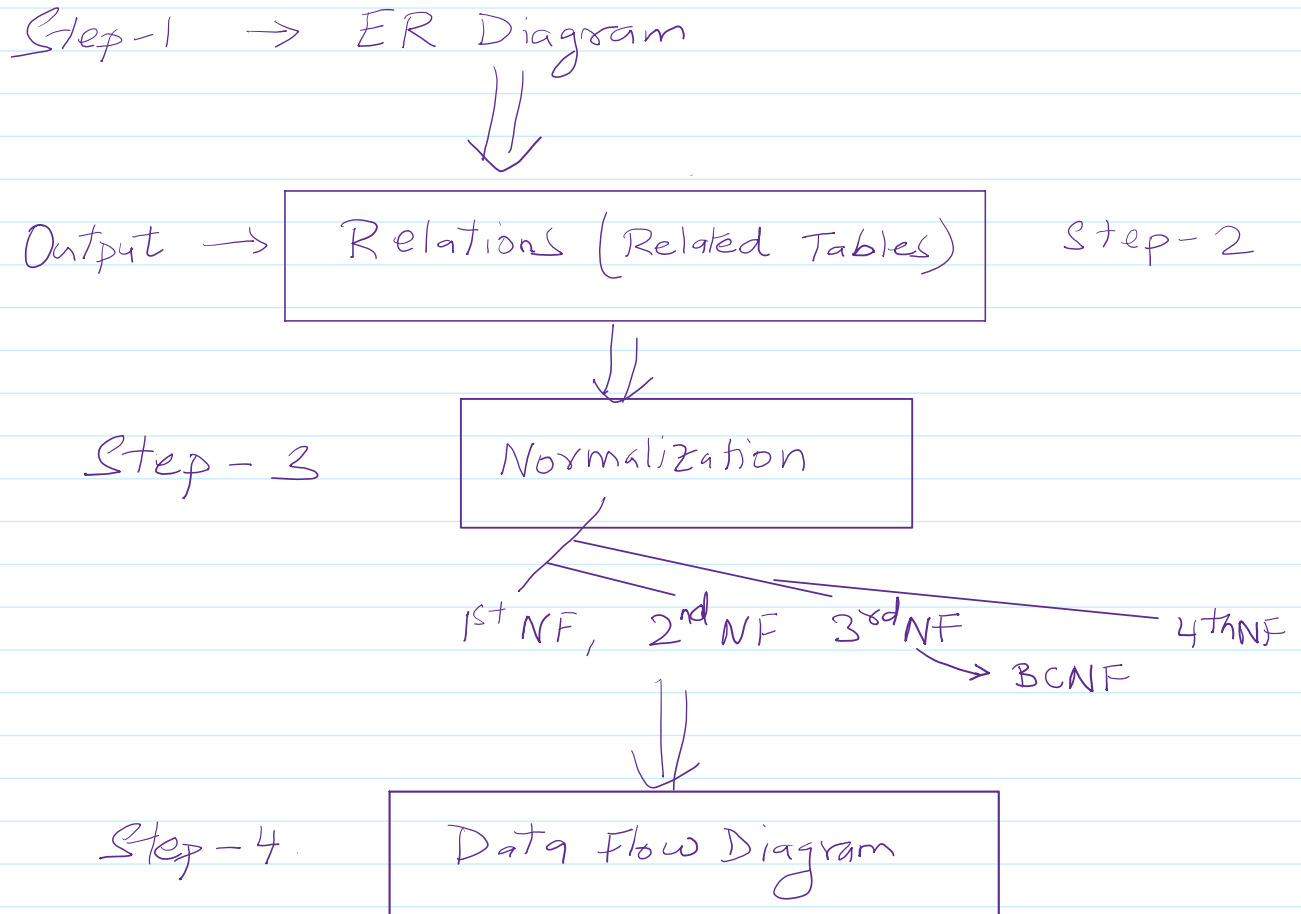


ER Diagrams

25 September 2026 19:06



ER Diagram

An ER diagram (Entity-Relationship Diagram) is a graphical representation of the data, entities, attributes, and relationships in a database.

It is mainly used during the database design phase to understand how data will be organized before creating tables in a DBMS such as Mysql, Oracle. Postgresql or SQL server, etc.

Why do we need an ER diagram?

Suppose we want to create a database for a college management system.

The college needs to store information about:

- Students
- Teachers
- Courses
- Departments
- Subjects

- Enrollments

Before creating database tables we need to understand:

- What information should be stored?
- What are important objects?
- What properties does each object have?
- How the object related?
- How many objects can participate in a relationship?

An ER diagram help us answer these questions visually.

For example:

STUDENT-----ENROLLS-----COURSE

This tells us that student enroll in course.

Basic Components of an ER Diagram

The major component of an ER diagram are:

- 1) Entity
- 2) Attribute
- 3) Relationship
- 4) Key attribute
- 5) Cardinality
- 6) Participation

1. Entity: An entity is a real world object or concept about which we want to store information.

Student
Teacher
Employee
Department
Course
Customer
Product
Bank Account

For example, in a college database:

"Student" Is an entity because we want to store information about students.

Entity Representation

In a traditional ER diagram, an entity is represented by a rectangle.

Student

Entity Set: An entity set is a collection of similar entities.

For example:

"Student" is an entity type/set, while individual students are entities.

Suppose a college has:

101	Aditya
102	Deeptanarayan
103	Soumodip

Each individual student is an entity, while all students together form the **Student entity set**.

2. Attribute: An attribute describes a property or characteristics of an entity. For example, a student can have:

Student

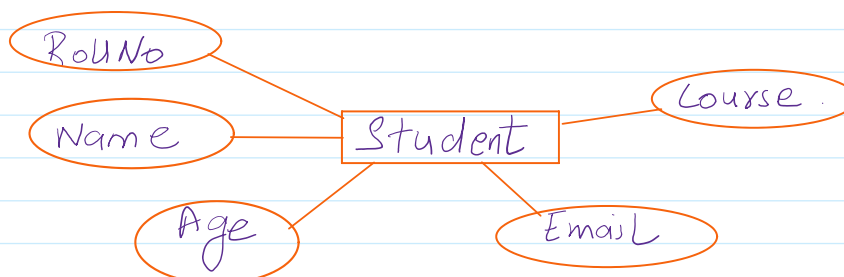
|
+--RollNo
+--Name
+--Age
+--Email
+--Course

Here:

- RollNo is an Attribute.
- Name is an Attribute.
- Age is an Attribute.
- Email is an Attribute.
- Course is an Attribute.

Representation

An attribute is traditionally represented by an Oval/ellipse.



Type of Attributes

There are several important types of attributes.

Simple Attribute

A simple attribute cannot be meaningfully divided into smaller attributes.

Example:

Age
Gender

Salary
RollNo

Composite Attribute

A composite attribute can be divided into smaller attributes.

For example:

"Name"

Can be divided into:

FirstName

MiddleName

LastName

Similarly:

"Address"

Could contain:

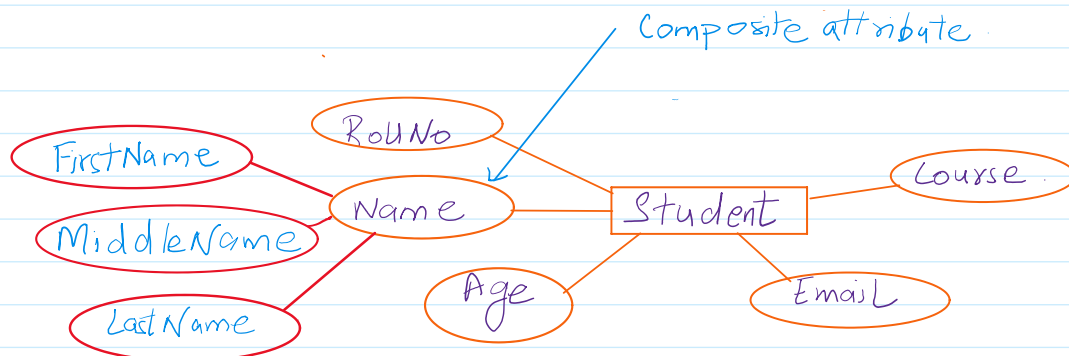
HouseNo

Street

City

State

Pincode



Single-Valued Attribute

A single - valued attribute has only one value for a particular entity.

Example:

Student -> DateOfBirth

A student normally has one date of birth.

Example:

RollNo

DateOfBirth

AdharNumber

EmployeeID

PassportNo

Multi-valued Attribute

A multi valued attribute can have multiple values for one entity.

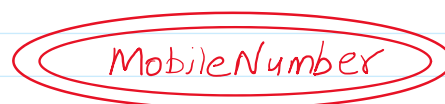
For example, an employee may have multiple mobile numbers:

Employee

|

+-9876545678

+-6345678233



In an ER diagram a multi valued attribute is represented using a double Oval.
Another example, a Student might have multiple emails.

Student

|

+-personal@gmail.com



Student

|

+---personal@gmail.com

+---college.edu@KRMangalam.edu

Email

Depending on the database requirements.

Derived Attributes

A derived attribute is an attribute whose values can be calculated from another attribute(s).

Example: DateOfBirth -> Age

If the date of birth is known, age can be calculated.

In an ER diagram, a derived attribute is represented using a dashed Oval.

Age

Another example:

(Salary + Bonus) -> totalSalary

totalSalary

Key Attributes: A key attribute uniquely identifies an entity.

For example, suppose we have:

Student

|

+--RollNo

+--Name

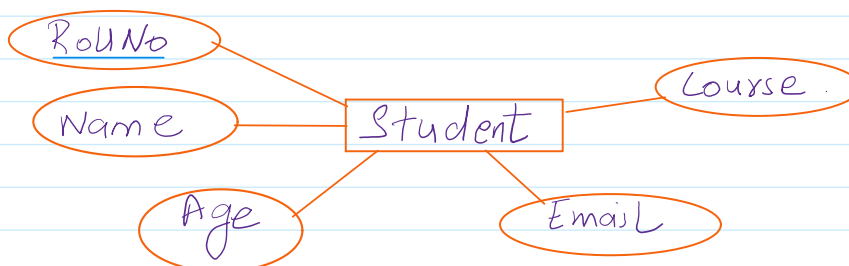
+--Age

+--Email

Two students could have the same name, but their **RollNo** should uniquely identify them. Therefore **RollNo** can be the primary key.

ER representation

A key attribute is traditionally represented by an underline attribute.



Here, 'RollNo' is underlined, Hence it is a Primary key.

Relationship

A relationship represents an association between two or more entities.

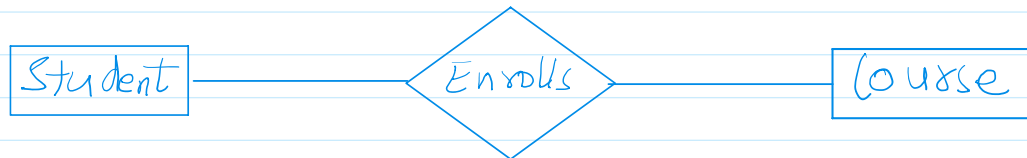
For example:

STUDENT ---- ENROLLS ----- Course

Here:

- STUDENT is an entity.
- COURSE is an entity.
- ENROLLS is a relationship.

In traditional ER diagram, a relationship is represented by a diamond.



ER relationship diagram

Relationship Name

A relationship should normally have a meaningful name.

Examples:

Entity1	Relationship	Entity2
Student	Enrolls	Course
Employee	Works_For	Department
Customer	Places	Order
Doctor	Treats	Patient
Teacher	Teaches	Subject
Customer	Purchases	Product

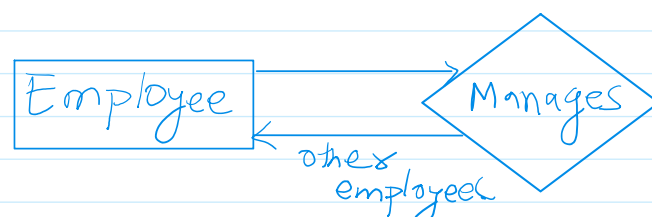
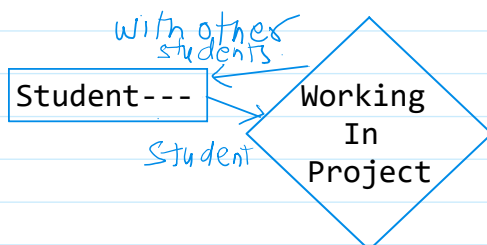
Degree of Relationship

The degree of a relationship specifies how many entity types participate in the relationship.

The major types are:

1. Unary
2. Binary
3. Ternary

1. Unary relationship: A unary relationship involves one entity type.



→ 10/10/11

Project

0.128
employee

The same entity participate in the relationship in different roles:
Manger is an Employee managing other Employee